Architecture Audit & Feasibility Report

BLoC Migration Assessment Report

Polzet Social Media Platform • Flutter / Dart Application

Date: October 2026
Status: Informational Only (No Changes Made)
Target Pattern: `flutter_bloc` + Clean Architecture

TOTAL DART FILES
260
Entire codebase in `lib/`

ACTIVE PROVIDER FILES
59
Direct `provider` imports

EXISTING CHANGENOTIFIERS
6
Chat, User, Profile, Themes, Net

DIRECT APISERVICE SCREENS
78
Screens with local `setState()`

1. Migration Scope Definition (2 Approaches)

Scope Level	Files Modified	New Files Created	Description & Effort
Level 1: Provider-to-BLoC Core RECOMMENDED PHASE 1	**~65 files**	**~18 – 24 files** (Events, States, Blocs)	Replaces the existing 6 ChangeNotifiers and updates all screens currently importing `package:provider`. Preserves existing screen API interactions.
Level 2: Full Project BLoC Architecture COMPREHENSIVE CLEAN ARCH	**~145 – 160 files**	**~50 – 70 files** (Blocs, Repos, Models)	Migrates all 78 screens that currently make raw API calls directly using `setState()` into decoupled BLoCs and Repositories (Auth, Feeds, Polls, Search).

2. Current State Management Inventory (To Be Converted)

Current File	Size	Target BLoC / Cubit Replacement	Complexity	Affected UI Consumers
`lib/provider/connection_provider.dart`	3.0 KB	`ConnectivityCubit` (Online / Offline / Restored)	LOW	ConnectivityOverlay, Main wrapper
`lib/core/themes/theme_provider.dart`	1.0 KB	`ThemeCubit` (Light / Dark mode toggle)	LOW	Theme settings, MaterialApp theme
`lib/provider/public_profile_provider.dart`	7.8 KB	`PublicProfileBloc` (Events: Fetch, Follow, Block)	MEDIUM	PublicProfileScreen, PublicChaseList
`lib/provider/user_provider.dart`	28.4 KB	`UserBloc` / `AuthProfileBloc`	HIGH	~28 screens (Profile, Settings, Feed Cards)
`lib/provider/private_chat_provider.dart`	31.0 KB	`PrivateChatBloc` (Socket & REST events)	HIGH	PrivateChatScreen, MessageList, UserInfo
`lib/provider/group_chat_provider.dart`	43.9 KB	`GroupChatBloc` (Socket, Group Members, Attachments)	HIGH	GroupChatScreen, 8 Group Info screens

3. Detailed Breakdown of UI & Screen Changes (59 Files)

Module / Feature	Files Count	Key Screen & Widget Files	Changes Needed
Direct Chat & Messaging	14 files	`private_chat_screen.dart` `group_chat_screen.dart` `message_list.dart` `archived_chats_screen.dart` `group_info_screen.dart`	Replace `Consumer<PrivateChatProvider>` with `BlocConsumer<PrivateChatBloc, PrivateChatState>`; dispatch socket events to BLoC.
User Profile & Account	9 files	`profile_screen.dart` `public_profile_screen.dart` `edit_profile.dart` `user_chase.dart`, `public_chase_list.dart`	Convert profile fetching and chase/follower button actions to BLoC events; replace `context.read<UserProvider>()`.
New Poll Creation	7 files	`new_text_poll.dart`, `new_image_poll.dart` `new_hot_take_poll.dart`, `new_battel_poll.dart` `new_anonymous_poll.dart`, `new_this_or_that.dart`	Replace user state reads and draft storage lookups with BLoC reads.
Settings & Security	8 files	`settings_screen.dart` `delete_account.dart` `private_account.dart`, `feedback.dart`	Switch theme toggle and account status modifications to BLoC event triggers.
Feed & Notifications	11 files	`home_feed_post_card.dart` `notification.dart` `poll_vote_notification_tile.dart` `single_post_details.dart`	Update like/vote dispatching from provider mutations to `context.read<Bloc>().add(Event)`.
App Lifecycle & Routing	6 files	`main.dart` `notification_router.dart` (6 ChangeNotifier instances) `connectivity_overlay.dart`	Replace top-level `MultiProvider` with `MultiBlocProvider`; remove route-level providers.

4. Architecture Transition: Current vs. BLoC

Current Provider Pattern

- Uses mutable `ChangeNotifier` with internal setters.
- Calls `notifyListeners()` which triggers full subtree rebuilds.
- UI screens call provider methods directly (e.g., `provider.fetchChatHistory()`).
- Socket listeners update internal list references in-place.
- 78 screens mix networking (`ApiService`) with UI `setState()`.

Target BLoC Pattern

- Strict separation: **Events** (inputs) → **States** (immutable outputs).
- UI dispatches events: `context.read<Bloc>().add(FetchChatEvent())`.
- Fine-grained rebuilds using `buildWhen` and `BlocBuilder`.
- Side-effects (SnackBars, navigation) handled via `BlocListener`.
- Zero raw API calls in UI screens; clean repository architecture.

5. Recommended Phased Implementation Roadmap

1 **Phase 1: Dependencies & Lightweight Cubits** — 5 FILES • SAFE START

Add `flutter_bloc: ^8.1.6` and `equatable: ^2.0.5` to `pubspec.yaml`. Convert `ConnectivityProvider` and `ThemeProvider` into Cubits. *Note:* Provider and BLoC can safely run concurrently in the app during this step.

2 **Phase 2: User Profile & Public Profile BLoCs** — ~35 FILES

Create `UserBloc` and `PublicProfileBloc`. Move profile loading, follow/unfollow, block/unblock logic into events. Refactor profile screens, edit profile, and poll creation screens to read from `UserState`.

3 **Phase 3: Real-Time Chat BLoCs (WebSocket Integration)** — ~25 FILES • CRITICAL

Convert `PrivateChatProvider` and `GroupChatProvider` into `PrivateChatBloc` and `GroupChatBloc`. Handle WebSocket event streams cleanly via Dart streams feeding into BLoC events (`MessageReceived` , `TypingStatusChanged` , `MessageSeen`).

4 **Phase 4: Navigation Clean-up & Provider Removal** — ~10 FILES

Clean up `notification_router.dart` route-level providers. Remove `provider` package dependency completely from `pubspec.yaml` once all 59 files are converted.

5 **Phase 5 (Optional): Screen-Level Feature BLoCs** — ~78 FILES

Gradually extract raw `ApiService` calls from screens (Feed, Comments, Poll Voting, Search, Analytics) into dedicated feature BLoCs (`FeedBloc`, `PollBloc`, etc.) to achieve 100% clean architecture across every screen.

6. Risk Matrix & Quality Assurance Recommendations

Risk Area	Severity	Mitigation Strategy
WebSocket Disconnection / Reconnection	HIGH	Maintain socket lifecycle in a singleton `WebSocketService` rather than recreating connections inside BLoC builders.
Route Navigation Context Loss	MEDIUM	Provide global BLoCs (`UserBloc` , `PrivateChatBloc`) at root in `main.dart` using `MultiBlocProvider` so child routes retain state.
Excessive Re-renders	LOW	Use `buildWhen: (previous, current) ⇒ ...` in `BlocBuilder` to prevent entire screen redraws when only a single counter or message updates.